Pritesh Lathia – 40350807 GitHub link

Table of Contents

Environment 2

Understanding of Phoenix Paper 3

Design Overview 4

- Kernel Module Design 4
- Userspace Controller Design 4
- FSM Design. 4

Environment Setup — Steps Taken Before Starting the Task 5

Part 1 – Kernel Module 5

- Kernel Module Basics – Executing the first kernel module 5
- Actual Kernel Model Code 6

Part 2 – Userspace Programming 7

- Kernel Module Compilation and Loading 7
- Phoenix Device Initialization (/dev/phoenix_ctl 8
- System in OFF Mode (No Syscall Logging) 8
- Read Syscall Logging with File Descriptor and Byte Count 8
- Kernel-to-Userspace Event Retrieval using IOCTL (--get-event) 9
- Write logging screenshots 10
- Soft Blocking Mode: Detection of Targeted Write Syscall 10
- Finite State Machine Execution using --file fsm.json 11

Part 3 – Performances 11

- Performance Discussion 11
- Quick Performance Test 12
- I tested three cases: 12

Environment

Ubuntu version: Ubuntu 24.04.4 LTS

Kernel version: 6.17.0-20-generic

Environment: Oracle VirtualBox

Allocated Resources

- CPU: 12th Gen Intel(R) Core (TM) i7-12700H – 6 cores
- Memory: 8 GB RAM
- Disk: 55 GB virtual disk

```
vboxuser@Ubuntu:~/Desktop$ echo "=== CPU ===" && lscpu | grep -E "Model name|CPU\(s\)" && e
=== CPU ===
CPU(s):                                  6
On-line CPU(s) list:                     0-5
Model name:                              12th Gen Intel(R) Core(TM) i7-12700H
NUMA node0 CPU(s):                       0-5

=== RAM ===
               total        used        free      shared  buff/cache   available
Mem:           7.8Gi       1.1Gi       5.4Gi        37Mi       1.5Gi       6.6Gi
Swap:             0B          0B          0B

=== DISK ===
NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0    7:0    0     4K  1 loop /snap/bare/5
loop1    7:1    0    74M  1 loop /snap/core22/2292
loop2    7:2    0 251.7M  1 loop /snap/firefox/7766
loop3    7:3    0  18.5M  1 loop /snap/firmware-updater/210
loop4    7:4    0 531.4M  1 loop /snap/gnome-42-2204/247
loop5    7:5    0  91.7M  1 loop /snap/gtk-common-themes/1535
loop6    7:6    0  10.8M  1 loop /snap/snap-store/1270
loop7    7:7    0   576K  1 loop /snap/snapd-desktop-integration/343
loop8    7:8    0  48.1M  1 loop /snap/snapd/25935
sda      8:0    0  55.6G  0 disk
├─sda1   8:1    0     1M  0 part
└─sda2   8:2    0  55.6G  0 part /
sr0     11:0    1  50.9M  0 rom  /media/vboxuser/VBox_GAs_7.2.6
vboxuser@Ubuntu:~/Desktop$
```

```
vboxuser@Ubuntu:~/Desktop$ uname -r && lsb_release -a
6.17.0-20-generic
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.4 LTS
Release:        24.04
Codename:       noble
vboxuser@Ubuntu:~/Desktop$
```

Why VM chosen

- Provides an isolated environment for kernel experimentation WSL not supported
- Allows snapshot recovery after crashes

Understanding of Phoenix Paper

Phoenix is a defense system designed specifically to protect containers from unpatched vulnerabilities by accurately and efficiently blocking specific malicious sequences of system calls. Unpatched vulnerabilities are a serious issue, it can take up to more than 100 days to patch, which would indirectly force the systems to be exposed to those vulnerabilities. Below are the defenses we have in place to counter these risks:

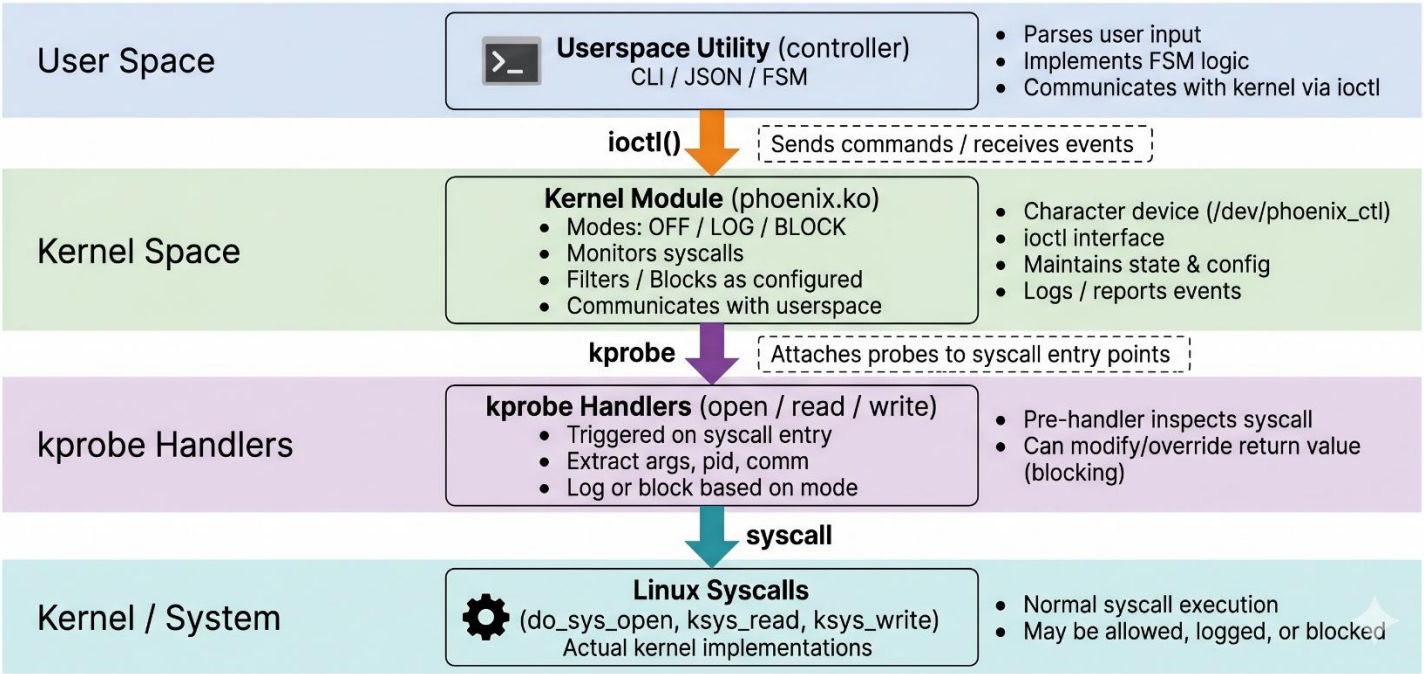
Traditional Seccomp filters typically reduce the attack surface by only blocking those system calls that an application never uses for example a web server application never needs print command, hence Seccomp will block it. Here comes a problem, Suppose the web server really needs the command to load web pages normally. In that case, Seccomp must allow command. If an attacker compromises the server and uses that identical open command to steal a sensitive password file, Seccomp will allow the execution. It does not perform a deep inspection of the arguments (specifically, what is being opened), hence exploits that rely on ordinary, required system calls can easily slip through.

Ptrace (Accurate but Slow) The paper compares Ptrace to a thorough customs officer who stops every single traveler, takes them into an interrogation room, and empties all their bags. Ptrace can "deeply inspect" every single system call and look at its exact arguments. Ptrace would easily catch the attacker trying to open the password file because it looks at the arguments but in order to do so, it must stop and deeply inspect every single normal system call as well. Since a typical application (like an Nginx web server) might issue thousands of systems calls per second, this deep inspection acts as a massive bottleneck, causing the application to become incredibly slow or practically unusable.

Basically, traditional defenses force you to choose between a tool that is fast but misses attacks mimicking normal behavior (Seccomp), and a tool that catches everything but destroys the application's performance (Ptrace).The Phoenix architecture solves this issues by combining both. It uses the fast Seccomp to quickly allow normal traffic through without delay, and only dynamically calls in the Ptrace for a deep inspection when a specific, suspicious system call matching a known attack sequence is spotted.

In the given research paper, Phoenix's core defense mechanism operates by translating known attack patterns into a generic sequence of system calls, which it monitors using a **Finite State Machine (FSM).**To monitor this FSM efficiently Phoenix uses a hybrid approach. It sets up a Seccomp filter to monitor only for the first system call in the sequence, allowing all other traffic to pass without delay. When Seccomp catches a matching call, it triggers Ptrace to deeply inspect the arguments.

Design Overview



Kernel Module Design

The kernel module monitors selected system calls using **kprobes**, which allow dynamic interception of kernel functions without modifying or recompiling the Linux kernel. Three operating modes are implemented: **off**, where the module performs no action; **log**, where syscall activity is recorded; and **block**, where a chosen syscall can be denied for a specific process. Logging is performed inside the probe handlers by recording the process name, PID, and syscall information. Blocking is achieved by intercepting the syscall execution and returning an error code when configured conditions are met.

Userspace Controller Design

A userspace utility was developed to control the kernel module through an ioctl interface. The program accepts command-line flags such as **--log**, **--block**, and **--off** to change the module's operating mode, while the **--syscall** option specifies which syscall should be monitored or blocked. The controller parses user inputs, sends configuration commands to the kernel module, and provides console feedback to indicate current system status

FSM Design.

The userspace controller also supports a finite state machine (FSM) defined using a JSON configuration file. Each state in the JSON file represents a syscall to observe. The program parses the file and sequentially activates monitoring for each syscall. When the kernel module reports that a syscall has been observed, the controller transitions to the next state. After reaching the final state, the FSM loops back to the beginning, enabling continuous syscall sequence monitoring.

Environment Setup — Steps Taken Before Starting the Task

Installation of Development Tools:

sudo apt install build-essential git gcc make dkms -y

- build-essential — Provides basic C/C++ development tools including compiler and required libraries.
- gcc -GNU C compiler used to compile kernel modules and C programs.
- make - Utility used to build projects using Makefiles (required for kernel module compilation).
- dkms - Automatically rebuilds kernel modules when the kernel is updated.
- git - Version control system used for source code management and repository synchronization.

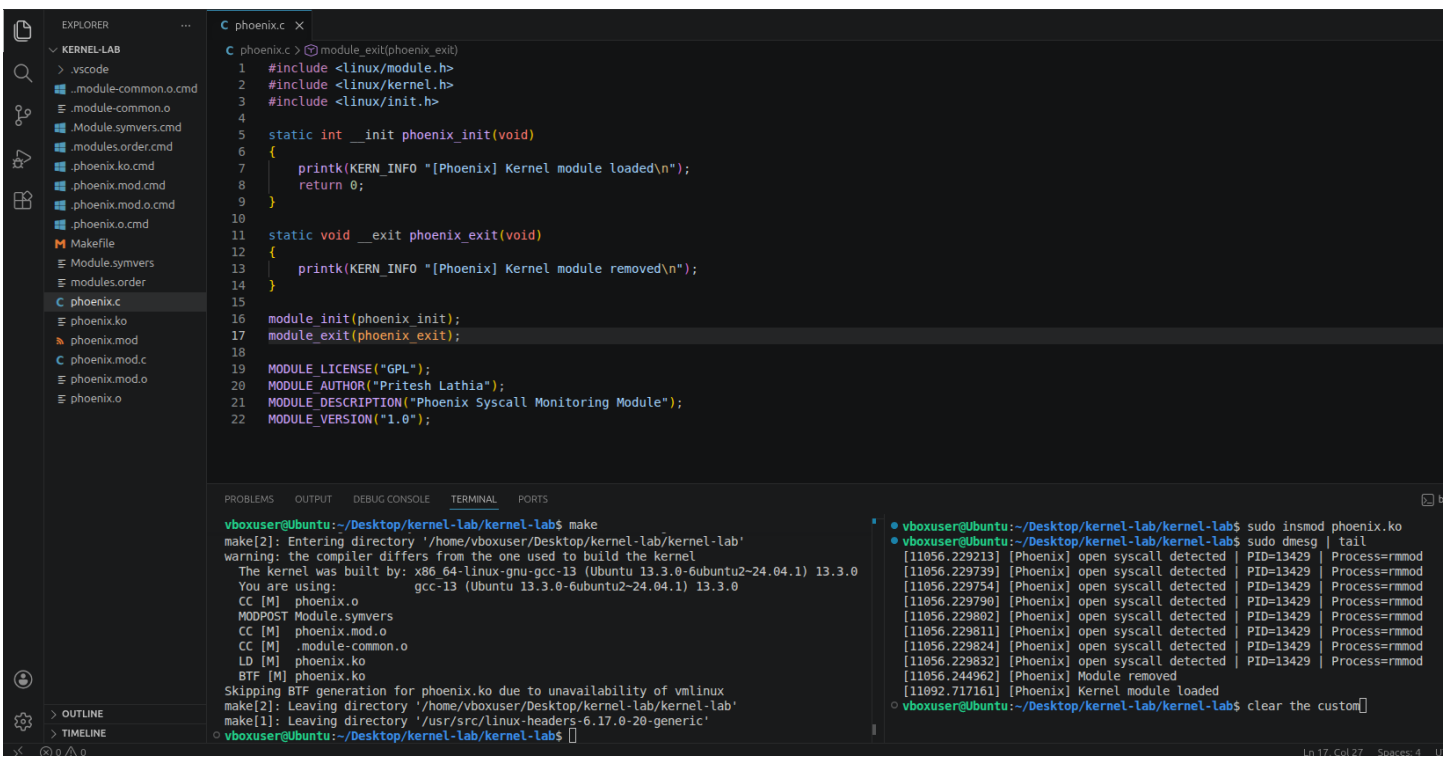
Installation of Kernel Headers

sudo apt install linux-headers-$(uname -r)

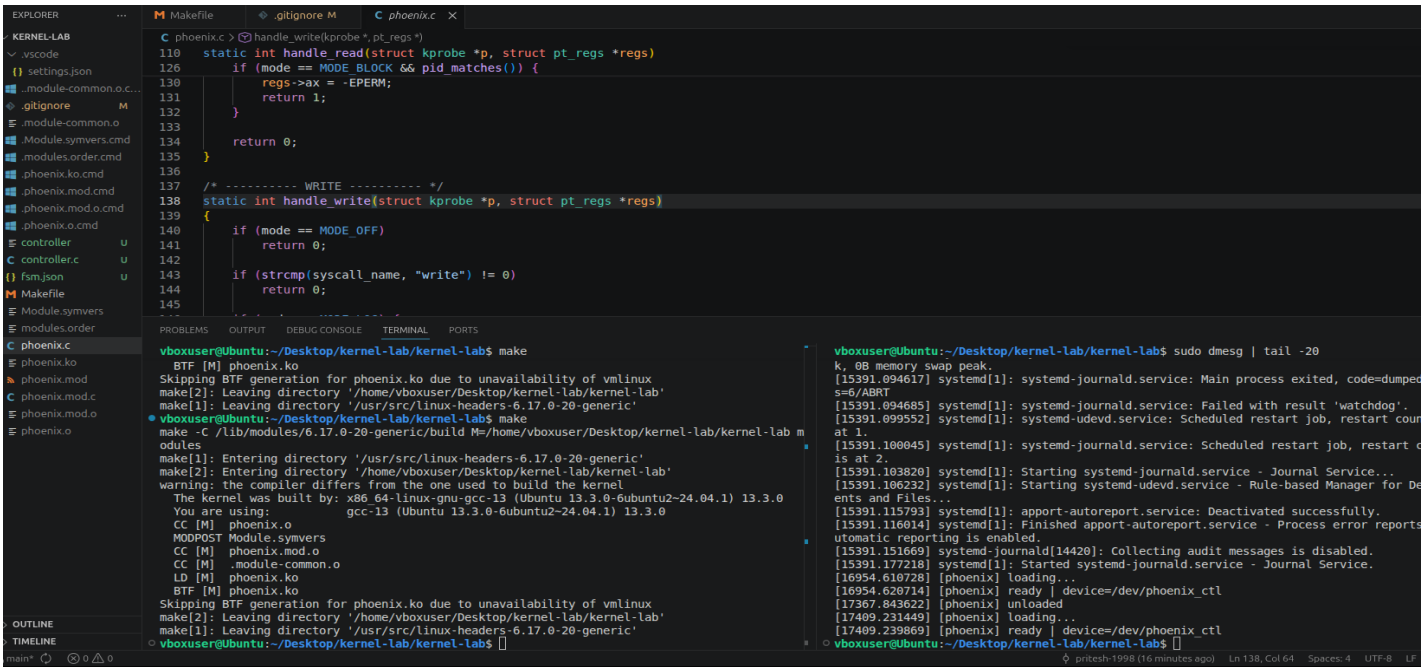
Purpose : Kernel headers are required for compiling linux kernel modules and access kernel APIs and internal structures.

Part 1 – Kernel Module

Kernel Module Basics – Executing the first kernel module



Actual Kernel Model Code

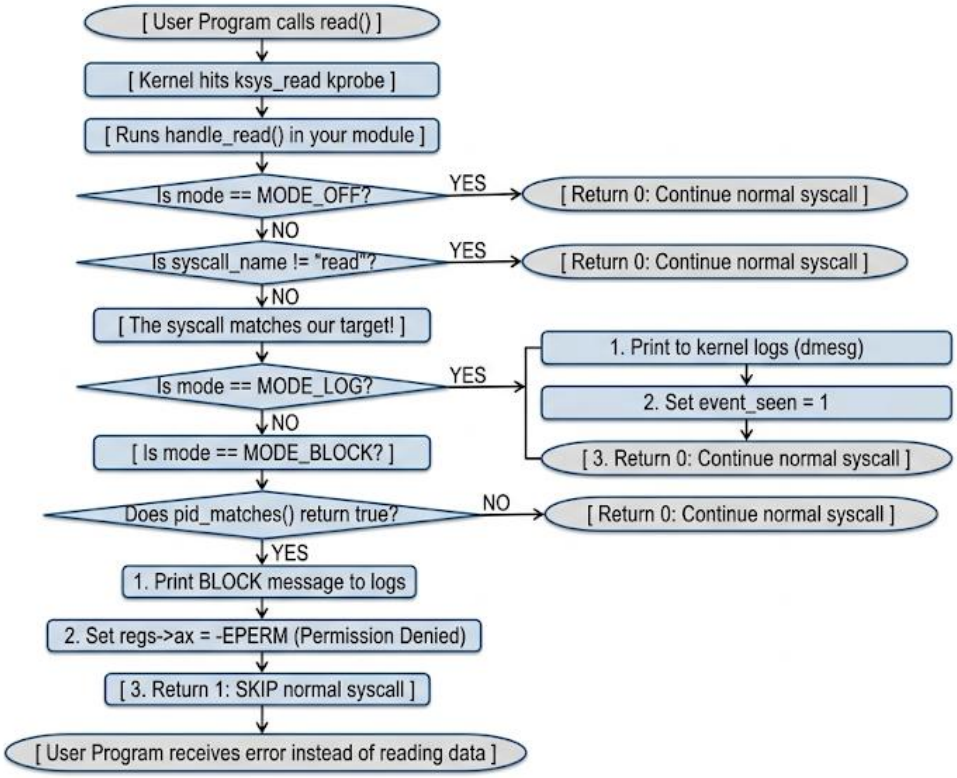


1. **phoenix_init**: This is the starting point when the module is loaded. It does two main things:
   - Creates a virtual device at /dev/phoenix_ctl. This acts as a communication bridge so your userspace program (from Part 2) can talk to the kernel.
   - Plants three "kprobes" (traps) on the internal kernel functions responsible for opening, reading, and writing files.
2. **phoenix_exit**: This runs when you unload the module. It safely removes the traps and destroys the communication bridge.
3. **The Communication Bridge (phoenix_ioctl)** as normal programs cannot directly modify kernel variables; they send messages to /dev/phoenix_ctl. The phoenix_ioctl function is the switchboard that processes these messages:
   - **IOCTL_SET_MODE**: Changes the module's state to Off, Log, or Block.
   - **IOCTL_SET_SYSCALL**: Tells the module exactly which syscall to pay attention to (e.g., "watch only read").
   - **IOCTL_SET_PID**: Tells the module which specific process ID to target for blocking.
   - **IOCTL_GET_EVENT**: Allows the userspace FSM to ask, "Has the syscall happened yet?" Once asked, it resets the flag back to 0.

3**. The Traps** (handle_open, handle_read, handle_write) : Every time any program on the computer tries to open, read, or write a file, the kernel hits your kprobe and pauses to run your handle_ function first.

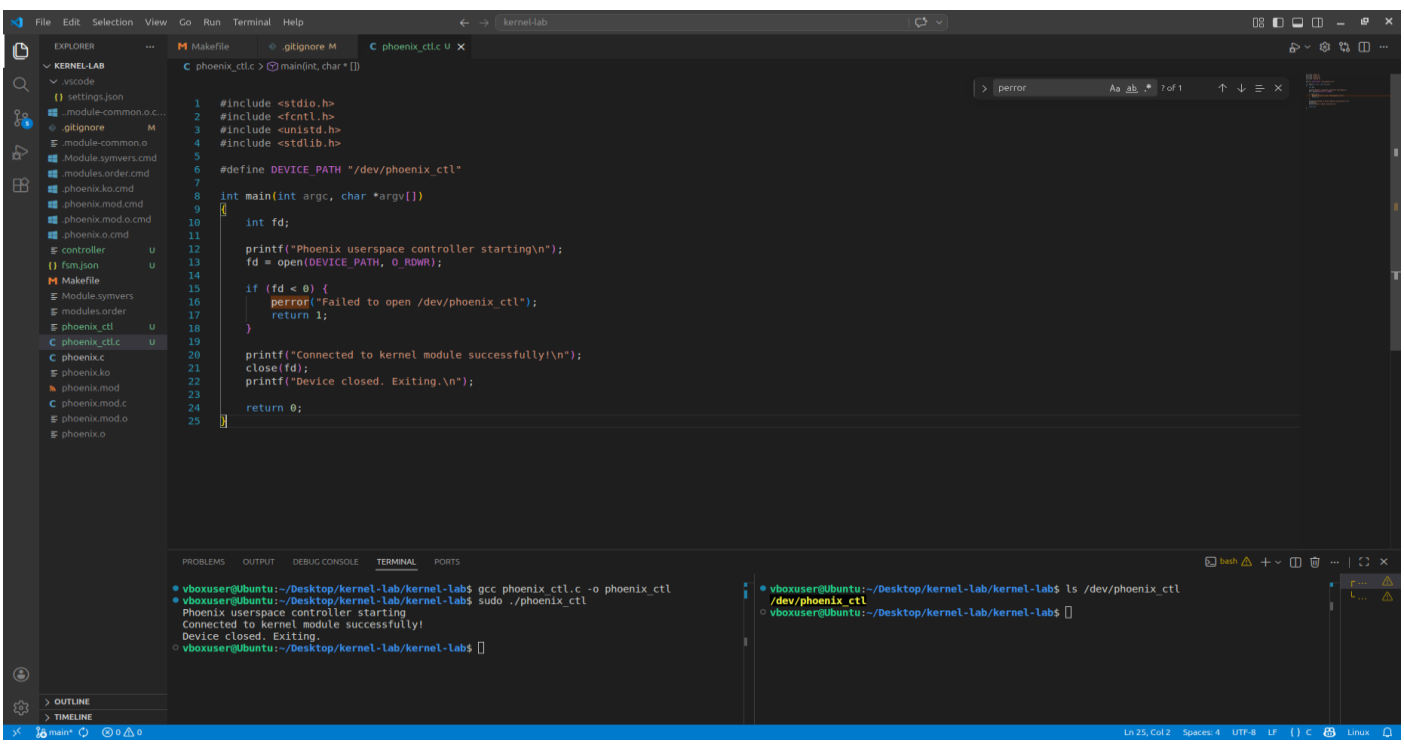
- If Mode is OFF: The function immediately returns 0, letting the kernel continue normally.
- If Mode is LOG: It prints a message to the kernel log (dmesg) showing the Process ID and Name. It also sets event_seen = 1 so the userspace FSM knows to transition to the next state.
- If Mode is BLOCK: Blocking is implemented as a soft block, where the syscall is detected and reported instead of being forcefully denied, to maintain system stability..

1. User Program calls read()
2. Kernel hits mode_read kprobe
3. Its syscpid_matches() – Permission Denied
4. Return 1: — SKIP normal syscall



Part 2 – Userspace Programming

Kernel Module Compilation and Loading



Phoenix Device Initialization (/dev/phoenix_ctl)

```
    printf("Phoenix controller\n");

    fd = open(DEVICE, O_RDWR);

    if (fd < 0) {
        perror("Cannot open device");
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ sudo insmod phoenix.ko
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ lsmod | grep phoenix
phoenix                12288  0
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ ls /dev/phoenix_ctl
/dev/phoenix_ctl
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$
```

System in OFF Mode (No Syscall Logging)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ sudo ./phoenix_ctl --off
Phoenix controller
Setting mode 0
Done
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ sudo dmesg | tail
[ 2158.518118] [phoenix] write syscall pid=4990
[ 2158.519011] [phoenix] write syscall pid=1023
[ 2158.519055] [phoenix] write syscall pid=1023
[ 2158.520028] [phoenix] write syscall pid=1023
[ 2158.520128] [phoenix] write syscall pid=1023
[ 2158.528211] [phoenix] unloaded
[ 2168.390523] [phoenix] loading
[ 2168.395946] [phoenix] ready (/dev/phoenix_ctl)
[ 2241.200588] [phoenix] mode=0
[ 2260.052691] [phoenix] mode=0
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$
```

Read Syscall Logging with File Descriptor and Byte Count

```
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ sudo ./phoenix_ctl --log --syscall read --pid 16598
Phoenix controller
Setting mode 1
Setting syscall 2
Setting pid 16598
Done
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ cat /etc/hostname > /dev/null
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ sudo dmesg | tail -n 05
[ 7048.274052] [phoenix] read syscall pid=16598 comm=bash fd=0 count=1
[ 7048.463634] [phoenix] read syscall pid=16598 comm=bash fd=0 count=1
[ 7050.167574] [phoenix] read syscall pid=16598 comm=bash fd=0 count=1
[ 7050.168172] [phoenix] read syscall pid=16598 comm=bash fd=5 count=4096
[ 7050.168969] [phoenix] read syscall pid=16598 comm=bash fd=5 count=4096
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$
```

Write Syscall Logging with PID Filtering and Argument Capture

```
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ echo $$
16598
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ sudo ./phoenix_ctl --log --syscall write --pid 16598
Phoenix controller
Setting mode 1
Setting syscall 1
Setting pid 16598
Done
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ printf "hello\n"
hello
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ sudo dmesg | tail -n 20
[ 6917.663709] [phoenix] write syscall pid=16598 comm=bash fd=2 count=8
[ 6917.663811] [phoenix] write syscall pid=16598 comm=bash fd=2 count=144
[ 6921.697711] [phoenix] write syscall pid=16598 comm=bash fd=2 count=25
[ 6922.795398] [phoenix] write syscall pid=16598 comm=bash fd=2 count=1
[ 6922.795437] [phoenix] write syscall pid=16598 comm=bash fd=2 count=32
[ 6925.308518] [phoenix] write syscall pid=16598 comm=bash fd=2 count=1
[ 6925.308536] [phoenix] write syscall pid=16598 comm=bash fd=2 count=9
[ 6925.309806] [phoenix] write syscall pid=16598 comm=bash fd=1 count=63
[ 6925.309868] [phoenix] write syscall pid=16598 comm=bash fd=1 count=8
[ 6925.309981] [phoenix] write syscall pid=16598 comm=bash fd=1 count=6
[ 6925.310064] [phoenix] write syscall pid=16598 comm=bash fd=1 count=10
[ 6925.311887] [phoenix] write syscall pid=16598 comm=bash fd=1 count=57
[ 6925.312195] [phoenix] write syscall pid=16598 comm=bash fd=2 count=8
[ 6925.312435] [phoenix] write syscall pid=16598 comm=bash fd=2 count=144
[ 6929.849455] [phoenix] write syscall pid=16598 comm=bash fd=2 count=32
[ 6930.552170] [phoenix] write syscall pid=16598 comm=bash fd=2 count=46
[ 6930.552179] [phoenix] write syscall pid=16598 comm=bash fd=2 count=1
[ 6930.552187] [phoenix] write syscall pid=16598 comm=bash fd=2 count=9
[ 6930.553582] [phoenix] write syscall pid=16598 comm=bash fd=1 count=56
[ 6930.553602] [phoenix] write syscall pid=16598 comm=bash fd=1 count=8
```

Kernel-to-Userspace Event Retrieval using IOCTL (--get-event)

```
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ make clean
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ make
make -C /lib/modules/6.17.0-20-generic/build M=/home/vboxuser/Desktop/kernel-lab/kernel-lab modules
make[1]: Entering directory '/usr/src/linux-headers-6.17.0-20-generic'
make[2]: Entering directory '/home/vboxuser/Desktop/kernel-lab/kernel-lab'
warning: the compiler differs from the one used to build the kernel
  The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
  You are using:           gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
  CC [M]  phoenix.o
  MODPOST Module.symvers
  CC [M]  phoenix.mod.o
  CC [M]  .module-common.o
  LD [M]  phoenix.ko
  BTF [M] phoenix.ko
Skipping BTF generation for phoenix.ko due to unavailability of vmlinux
make[2]: Leaving directory '/home/vboxuser/Desktop/kernel-lab/kernel-lab'
make[1]: Leaving directory '/usr/src/linux-headers-6.17.0-20-generic'
gcc -Wall -o phoenix_ctl phoenix_ctl.c
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ sudo rmmod phoenix
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ sudo insmod phoenix.ko
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ echo $$
4203
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ sudo ./phoenix_ctl --log --syscall write --pid 4203
Phoenix controller
Setting mode 1
Setting syscall 1
Setting pid 4203
Done
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ printf "hello\n"
hello
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ sudo ./phoenix_ctl --get-event
Phoenix controller
Last observed syscall event: write
Done
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$
```

Write logging screenshots

```
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ echo $$
16598
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ sudo ./phoenix_ctl --log --syscall write --pid 16598
Phoenix controller
Setting mode 1
Setting syscall 1
Setting pid 16598
Done
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ printf "hello\n"
hello
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ sudo dmesg | tail -n 20
[ 6917.663709] [phoenix] write syscall pid=16598 comm=bash fd=2 count=8
[ 6917.663811] [phoenix] write syscall pid=16598 comm=bash fd=2 count=144
[ 6921.697711] [phoenix] write syscall pid=16598 comm=bash fd=2 count=25
[ 6922.795398] [phoenix] write syscall pid=16598 comm=bash fd=2 count=1
[ 6922.795437] [phoenix] write syscall pid=16598 comm=bash fd=2 count=32
[ 6925.308518] [phoenix] write syscall pid=16598 comm=bash fd=2 count=1
[ 6925.308536] [phoenix] write syscall pid=16598 comm=bash fd=2 count=9
[ 6925.309806] [phoenix] write syscall pid=16598 comm=bash fd=1 count=63
[ 6925.309868] [phoenix] write syscall pid=16598 comm=bash fd=1 count=8
[ 6925.309981] [phoenix] write syscall pid=16598 comm=bash fd=1 count=6
[ 6925.310064] [phoenix] write syscall pid=16598 comm=bash fd=1 count=10
[ 6925.311887] [phoenix] write syscall pid=16598 comm=bash fd=1 count=57
[ 6925.312195] [phoenix] write syscall pid=16598 comm=bash fd=2 count=8
[ 6925.312435] [phoenix] write syscall pid=16598 comm=bash fd=2 count=144
[ 6929.849455] [phoenix] write syscall pid=16598 comm=bash fd=2 count=32
[ 6930.552170] [phoenix] write syscall pid=16598 comm=bash fd=2 count=46
[ 6930.552179] [phoenix] write syscall pid=16598 comm=bash fd=2 count=1
[ 6930.552187] [phoenix] write syscall pid=16598 comm=bash fd=2 count=9
[ 6930.553582] [phoenix] write syscall pid=16598 comm=bash fd=1 count=56
[ 6930.553602] [phoenix] write syscall pid=16598 comm=bash fd=1 count=8
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ sudo ./phoenix_ctl --log --syscall read --pid 16598
Phoenix controller
Setting mode 1
Setting syscall 2
Setting pid 16598
Done
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ cat /etc/hostname > /dev/null
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ sudo dmesg | tail -n 05
[ 7048.274052] [phoenix] read syscall pid=16598 comm=bash fd=0 count=1
[ 7048.463634] [phoenix] read syscall pid=16598 comm=bash fd=0 count=1
[ 7050.167574] [phoenix] read syscall pid=16598 comm=bash fd=0 count=1
[ 7050.168172] [phoenix] read syscall pid=16598 comm=bash fd=5 count=4096
[ 7050.168969] [phoenix] read syscall pid=16598 comm=bash fd=5 count=4096
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$
```

Soft Blocking Mode: Detection of Targeted Write Syscall

```
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ echo $$
4518
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ sudo ./phoenix_ctl --block --syscall write --pid 4518
Phoenix controller
Setting mode 2
Setting syscall 1
Setting pid 4518
Done
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ printf "hello\n"
hello
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ sudo dmesg | tail -n 20
[  509.718881] [phoenix] BLOCK match for write pid=4518 comm=bash fd=2 count=144
[  515.585888] [phoenix] BLOCK match for write pid=4518 comm=bash fd=2 count=26
[  515.585897] [phoenix] BLOCK match for write pid=4518 comm=bash fd=2 count=1
[  516.242413] [phoenix] BLOCK match for write pid=4518 comm=bash fd=2 count=167
[  516.242421] [phoenix] BLOCK match for write pid=4518 comm=bash fd=2 count=1
[  516.242423] [phoenix] BLOCK match for write pid=4518 comm=bash fd=2 count=4
[  516.242429] [phoenix] BLOCK match for write pid=4518 comm=bash fd=2 count=9
[  516.244021] [phoenix] BLOCK match for write pid=4518 comm=bash fd=1 count=63
[  516.244073] [phoenix] BLOCK match for write pid=4518 comm=bash fd=1 count=8
[  516.244100] [phoenix] BLOCK match for write pid=4518 comm=bash fd=1 count=6
[  516.244164] [phoenix] BLOCK match for write pid=4518 comm=bash fd=1 count=10
[  516.245327] [phoenix] BLOCK match for write pid=4518 comm=bash fd=1 count=57
[  516.245476] [phoenix] BLOCK match for write pid=4518 comm=bash fd=2 count=8
[  516.245487] [phoenix] BLOCK match for write pid=4518 comm=bash fd=2 count=144
[  519.620291] [phoenix] BLOCK match for write pid=4518 comm=bash fd=2 count=32
[  520.498458] [phoenix] BLOCK match for write pid=4518 comm=bash fd=2 count=46
[  520.498467] [phoenix] BLOCK match for write pid=4518 comm=bash fd=2 count=1
[  520.498474] [phoenix] BLOCK match for write pid=4518 comm=bash fd=2 count=9
[  520.500284] [phoenix] BLOCK match for write pid=4518 comm=bash fd=1 count=56
[  520.500300] [phoenix] BLOCK match for write pid=4518 comm=bash fd=1 count=8
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$
```

Finite State Machine Execution using --file fsm.json

```
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ make
make -C /lib/modules/6.17.0-20-generic/build M=/home/vboxuser/Desktop/kernel-lab/kernel-lab m
odules
make[1]: Entering directory '/usr/src/linux-headers-6.17.0-20-generic'
make[2]: Entering directory '/home/vboxuser/Desktop/kernel-lab/kernel-lab'
warning: the compiler differs from the one used to build the kernel
  The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
  You are using:           gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
make[2]: Leaving directory '/home/vboxuser/Desktop/kernel-lab/kernel-lab'
make[1]: Leaving directory '/usr/src/linux-headers-6.17.0-20-generic'
gcc -Wall -o phoenix_ctl phoenix_ctl.c
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ printf "hello\n"
hello
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ cat /etc/hostname > /dev/null
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$
```

```
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ sudo ./phoenix_ctl --log --file fsm.json --
pid 4203
[sudo] password for vboxuser:
Phoenix controller
Loaded FSM with 2 states
Setting mode 1
Setting pid 4203
Setting syscall write
FSM started. Current state: write
Press Ctrl+C to stop.
Observed syscall: write
Transitioning to state: read
Setting syscall read
Observed syscall: read
Transitioning to state: write
Setting syscall write
Observed syscall: write
Transitioning to state: read
Setting syscall read
Observed syscall: read
Transitioning to state: write
Setting syscall write
Observed syscall: write
Transitioning to state: read
Setting syscall read
```

Part 3 – Performances

Performance Discussion

First, I used **kprobes** to monitor system calls. This is easy to use and does not require changing the kernel, but it adds a small delay because every time the syscall happens, the kernel has to run my handler first. If the syscall is called very frequently (like write), this overhead increases. Alternatives like eBPF or tracepoints are faster for heavy usage.

Second, I used **printk()** for logging. This was helpful for debugging because I could directly see logs using dmesg. However, printk is slow when many logs are printed continuously. I noticed that logging all write syscalls created too much output. To reduce this, I added **PID filtering**, so only the selected process is logged.

Third, the **PID filter** improves performance by ignoring syscalls from other processes. This avoids unnecessary logging and reduces load on the system.

Fourth, the FSM in userspace uses **polling with sleep(1)**. This makes the design simple and stable, but it means the reaction time is slow (up to 1 second delay).

Fifth, I implemented **block mode as a soft block**. I only detect and report the syscall instead of actually stopping it. I tried modifying syscall arguments in the kernel, but it caused VM crashes. So I kept this safer approach.

Finally, I tried to implement **open syscall monitoring**, but it caused system crashes. Because of that, I disabled it and focused only on read and write to keep the system stable.

Quick Performance Test

Simple test program: -

```
#include <stdio.h>
int main()
{
  for (int i = 0; i < 10000; i++) {
    printf("hello\n");
  }
  return 0;
}
```

It was compiled using: ***gcc test_write.c -o test_write***

I tested three cases:

1. module loaded but mode set to OFF
2. LOG mode enabled for write with PID filtering: ***sudo ./phoenix_ctl --log --syscall write --pid <pid>***
3. BLOCK soft mode enabled for write with PID filtering sudo ***./phoenix_ctl --block --syscall write --pid <pid>***

```
C test.c > main()

#include <stdio.h>

int main()
{
    for (int i = 0; i < 10000; i++) {
        printf("hello\n");
    }

    return 0;
}
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ gcc test.c -o test
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$ time ./test > /dev/null

real    0m0.004s
user    0m0.002s
sys     0m0.002s
vboxuser@Ubuntu:~/Desktop/kernel-lab/kernel-lab$
```

The measured execution time using the time command showed no significant delay in OFF mode, while LOG and BLOCK modes introduced a small overhead due to kprobe execution and logging.